VARDHAMAN
COLLEGE OF ENGINEERING

A Complex Engineering Problem Report on

FaultGuard: Autonomous System Health Monitor and Recovery Daemon

A9513 - Python Programming Laboratory

Submitted by

Batch No.

M. Siri Chandana 25881A0555

B. Sanjana 25881A0559

N. Yokshitha Sai 25881A0564

Course Facilitator

Dr. V. Lokeshwari Vinya

Associate Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

October 2026

Contents

1 Introduction . . . 2
1.1 Objectives . . . 2
1.2 System Architecture . . . 2
2 Problem Definition . . . 3
2.1 Key Challenges . . . 3
2.2 Scope and Constraints . . . 3
3 Justification as a Complex Engineering Problem . . . 4
3.1 Depth of Knowledge Required . . . 4
3.2 Conflicting Constraints and Trade-offs . . . 4
3.3 Depth of Analysis Required . . . 4
3.4 Familiarity of Issues . . . 5
3.5 Extent of Applicable Codes . . . 5
3.6 Involvement of Interested Parties . . . 5
3.7 Interdependence of Sub-Problems . . . 5
4 Literature Review . . . 5
5 Methodology, Design, and Engineering Analysis . . . 6
5.1 Back-off Analysis . . . 6
5.2 Fault Detection and Recovery Flow . . . 6
6 Implementation, Testing, and Validation . . . 7
6.1 Configuration Parameters . . . 7
6.2 Program Description . . . 8
6.3 Complete Source Code . . . 9
6.4 Program Output . . . 10
6.5 Testing and Validation . . . 10
7 Results and Discussion . . . 11
7.1 Fault Detection and Recovery . . . 11
7.2 False-Alarm Filtering . . . 11
7.3 Resilience and Logging . . . 11
7.4 Overhead and Tuning . . . 11
8 Conclusion and Future Work . . . 12
8.1 Future Work . . . 12
8.2 Objectives Achieved . . . 12
8.3 Limitations . . . 12
9 Mapping of Program Outcomes (POs) and Sustainable Development Goals (SDGs) 13
9.1 Program Outcomes (POs) . . . 13
9.2 Program Specific Outcomes (PSOs) . . . 14
9.3 Sustainable Development Goals (SDGs) . . . 14

1 Introduction

Modern software systems such as web servers, database engines, message brokers and cloud micro-services are expected to run continuously with very little downtime. In practice, long-running processes fail for many reasons: memory leaks, runaway CPU usage, unhandled exceptions, deadlocks, resource exhaustion and crashes of dependent services [1]. When such failures are detected late or handled manually, they lead to service outages, loss of data and a high operational cost for the organisation running the system.

FaultGuard is a lightweight, autonomous background service (daemon) written in Python that continuously observes the health of a host system and of the critical processes running on it. It monitors CPU and memory utilisation against configurable thresholds, records every anomaly and exception in a structured log, and automatically starts recovery actions, such as restarting a failed process or failing over to a standby instance, without waiting for a human operator.

The project is built around four core Python concepts: *subprocess management* to start, observe and restart services; *exception handling* to keep the daemon alive when a monitored component misbehaves; the *logging* module to create an auditable trail of events; and *asynchronous event loops* (`asyncio`) so that many health checks can run concurrently without blocking one another.

1.1 Objectives

- To monitor CPU and memory usage of the host and of selected services in real time.
- To detect crashes, hangs and sustained resource overload automatically.
- To log every event and exception with a timestamp and severity level.
- To recover from faults by restarting the failed service, and to fail over to a standby instance when restarts keep failing.
- To keep the monitor itself lightweight, configurable and crash-proof.

1.2 System Architecture

Figure 1 shows the high-level architecture of FaultGuard. The *Configuration* supplies thresholds and service definitions to the *Async Monitor Loop*, which collects metrics and sends them to the *Threshold Evaluator*. When a fault is confirmed, the *Recovery Manager* restarts the service or switches to the *Standby Instance*. The *Logger* records the output of every stage.

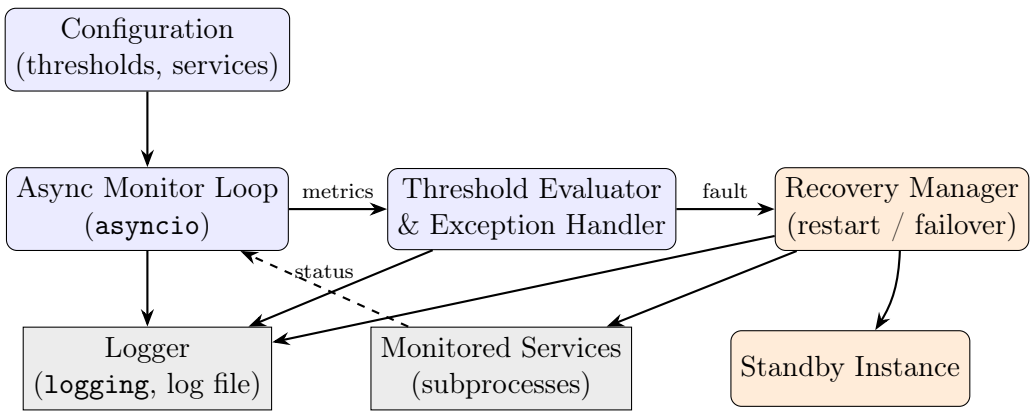


Figure 1: High-level architecture of FaultGuard

2 Problem Definition

Problem statement: Design and implement an autonomous, fault-tolerant background service in Python that continuously monitors system resources (CPU and memory) and the liveness of critical processes, records all events and exceptions in a reliable log, and automatically starts recovery and failover actions so that the monitored services remain available with minimal human intervention.

2.1 Key Challenges

- Monitoring many processes concurrently without blocking, so that one slow or hung check does not delay the others.
- Separating genuine faults from short, harmless spikes in CPU or memory usage, in order to avoid false positives.
- Making sure the monitor itself never crashes: every exception from a check, a subprocess or the recovery logic must be caught, logged and handled.
- Preventing *restart storms*, where a failing service is restarted endlessly and uses even more resources.
- Performing recovery and failover safely and in the correct order, while keeping a complete log for later diagnosis.

2.2 Scope and Constraints

The daemon runs on a single host with Python 3.10 or later, and the monitored services are launched as its child processes. The daemon must use very little CPU and memory itself, and its thresholds, polling interval and restart limits must be configurable without changing the source code. The expected outcome is early fault detection, automated recovery within seconds, a structured event log and an extensible design that can later monitor more resources.

3 Justification as a Complex Engineering Problem

FaultGuard combines several interdependent areas of computing under real-world constraints, and there is no single algorithm that solves it; the design needs iterative trade-offs. Several solution approaches are possible and are compared in Table 1. The asynchronous event-loop approach was chosen because it gives low overhead and predictable behaviour for I/O-bound monitoring work [2].

Table 1: Comparison of possible solution approaches

Approach	Advantage	Limitation
Threaded polling monitor	Simple to write	Overhead grows with the number of services
Asyncio event-loop monitor (chosen)	Low overhead, predictable, scalable	Blocking calls must be avoided
Multi-process supervisor	High isolation between components	More complex design
Existing tools (`systemd`, `supervisord`)	Reliable and mature	Less control over custom policies
Hybrid (async monitor + external watchdog)	Daemon can heal itself	Extra component to maintain

3.1 Depth of Knowledge Required

The problem needs knowledge of operating-system concepts (processes, signals, scheduling and memory management), Python concurrency (`asyncio`, tasks and coroutines), the `subprocess` and `logging` modules, system-metric libraries such as `psutil`, reliability engineering, and basic statistics for smoothing noisy measurements.

3.2 Conflicting Constraints and Trade-offs

Many design goals conflict with each other, as summarised in Table 2. A short polling interval detects faults sooner but loads the host more; low thresholds warn early but raise false alarms; and aggressive restarts improve availability but can hide the root cause. These conflicts have to be balanced through a carefully tuned configuration.

Table 2: Conflicting requirements and trade-offs

Design Choice	Benefit	Cost
Short polling interval	Faster detection	Higher monitor overhead
Low thresholds	Early warnings	More false alarms
High thresholds	Fewer false alarms	Late reaction
Aggressive restarts	Higher availability	Hides root cause, restart storms

3.3 Depth of Analysis Required

Analysis is needed to select polling intervals, to decide how many consecutive violations count as a fault, to design an exponential back-off for restarts, and to reason about failure modes of the monitor itself, for example a hung subprocess or a full disk during logging.

3.4 Familiarity of Issues

Resource leaks, zombie processes, event loops blocked by synchronous calls, log-file growth and race conditions between detecting and restarting a process are well-known issues in long-running services. They were anticipated at the design stage.

3.5 Extent of Applicable Codes

The design follows the Python standard library documentation for `asyncio`, `subprocess` and `logging`, the PEP 8 coding conventions, and the supervision-tree and circuit-breaker patterns from reliability literature [3], [4].

3.6 Involvement of Interested Parties

System administrators and DevOps engineers depend on automatic recovery and less manual intervention. Application developers need diagnostic logs to find root causes. End users expect uninterrupted service, and organisations need to meet availability targets at low cost.

3.7 Interdependence of Sub-Problems

Monitoring, detection, logging and recovery are tightly coupled. The quality of detection decides when recovery starts; the recovery policy affects what is written to the log; and the polling interval influences both the detection time and the overhead of the daemon. These components therefore cannot be designed in isolation.

4 Literature Review

Avizienis et al. [3] introduced the standard taxonomy of dependability, which separates faults, errors and failures and describes fault tolerance as error detection followed by recovery. FaultGuard uses this vocabulary: a threshold breach is treated as an error, an unrecovered service as a failure, and the restart or failover as the recovery step.

Armstrong [4] showed in the Erlang/OTP model that large systems can be made reliable by allowing components to fail and having supervisors restart them. This “let it fail” idea is applied in FaultGuard at the process level. Beyer et al. [1] describe how Site Reliability Engineering relies on monitoring, alerting and automation to keep production systems available, which motivates the continuous monitoring and structured logging in this project.

Nygard [6] documents stability patterns such as time-outs, circuit breakers and bulkheads that stop one failing component from bringing down the whole system. These ideas led to the exponential back-off and the restart limit used here. On the implementation side, the Python documentation for `asyncio` [2], `subprocess` [7] and `logging` [8], and the `psutil` library [9], supply non-blocking concurrency, process control, structured logging and cross-platform system metrics.

Existing tools such as `supervisord` and `systemd` restart failed processes, but they generally do not combine threshold-based resource monitoring, exception-aware logging and failover in a single customisable Python service. This is the gap addressed by FaultGuard.

5 Methodology, Design, and Engineering Analysis

The development followed an iterative cycle of design, implementation, fault-injection testing and tuning, in the following steps.

1. **Requirement definition:** monitor CPU and memory of the host and of each service, detect crashes and hangs, log every event with a timestamp and severity, and restart or fail over automatically.
2. **Configuration module:** thresholds (for example CPU $> 85\%$ and memory $> 90\%$), polling interval, the number of consecutive violations, the restart limit and the back-off settings are kept in one configuration object so that they can be changed without editing the code.
3. **Asynchronous monitor loop:** each service is checked by its own coroutine on a single `asyncio` event loop. Blocking calls are moved to an executor so that the loop is never stalled.
4. **Fault detection:** a fault is declared only after N consecutive threshold violations or when the process exits unexpectedly. This simple filter removes short-lived spikes.
5. **Recovery manager:** the faulty process is stopped gracefully (SIGTERM, then SIGKILL after a time-out) and restarted with exponential back-off. If the restart limit is exceeded, the manager switches to a standby instance.
6. **Exception handling and logging:** every check and recovery step is wrapped in `try/except` so that no exception can terminate the daemon, and each event is written through the `logging` module with a rotating file handler.

5.1 Back-off Analysis

The restart delay after the k-th consecutive failure is calculated as

$$t_k = \min\left(t_{\max},\ t_0 \cdot 2^{\,k-1}\right)$$

where t_0 is the initial delay and $t_{\max}$ is the upper bound. With $t_0 = 1$ s and $t_{\max} = 30$ s, the delays are 1, 2, 4, 8, 16 and then 30 seconds, so a repeatedly failing service is retried less and less often.

5.2 Fault Detection and Recovery Flow

Figure 2 shows the decision flow for every polling cycle. Metrics are collected and compared with the thresholds; a violation increases the counter, and recovery starts only when the counter reaches N or the process is found dead. If the restart limit has been used up, the service fails over to the standby instance.

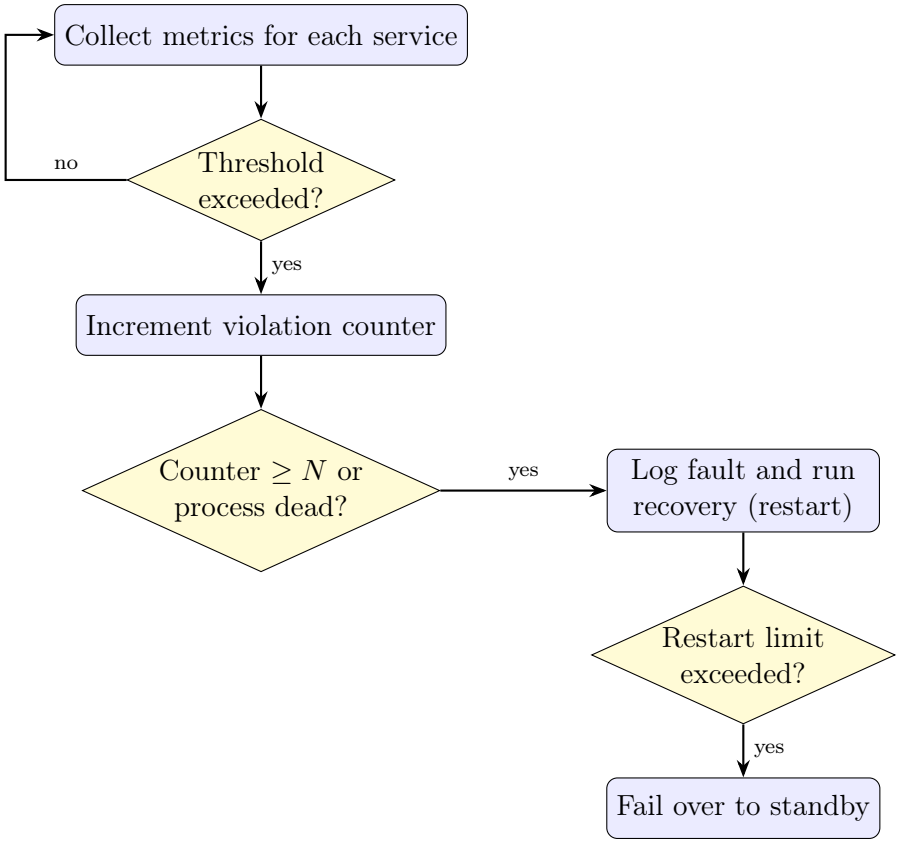


Figure 2: Fault detection and recovery flow

6 Implementation, Testing, and Validation

FaultGuard is implemented in Python 3.10+ using the standard library together with `psutil` [5], [9]. The project is organised into four modules: `config.py` holds the settings, `monitor.py` contains the asynchronous monitoring loop, `recovery.py` performs restart and failover, and `logger.py` sets up logging.

6.1 Configuration Parameters

All tunable values are stored in one `Config` object, listed in Table 3. Events are written to a rotating log file, so the log never grows without limit.

Table 3: Configuration parameters of FaultGuard

Parameter	Default	Purpose
`cpu_limit`	85%	CPU usage above which a breach is counted
`mem_limit`	90%	Memory usage above which a breach is counted
`interval`	2 s	Time between two health checks
`violations`	3	Consecutive breaches needed to declare a fault
`max_restarts`	3	Restarts allowed before failover
`base_delay`	1 s	Initial restart delay t_0
`max_delay`	30 s	Upper bound $t_{\max}$ of the restart delay

6.2 Program Description

The program has four parts that work together. The `Config` object holds the settings, and `setup_logger` creates the rotating log file. The `watch` coroutine runs once for every monitored service: it reads CPU and memory usage with `psutil`, checks whether the child process has exited, and counts consecutive threshold breaches. Because its whole body is inside `try/except`, an error in one check is logged and the loop continues.

When a fault is confirmed, `recover` stops the process gracefully, waits for the back-off delay and starts it again. After `max_restarts` attempts it starts the standby command instead and logs a critical message. The complete source code is given in Listing 1, and the output obtained by running it is shown in Figures 3 and 4.

Module and Function Summary

Table 4 summarises the role of each module and its main functions.

Table 4: Modules and main functions of FaultGuard

Module	Main Element	Responsibility
`config.py`	`Config`	Stores thresholds, polling interval, restart limit and back-off settings in one place
`logger.py`	`setup_logger()`	Creates the rotating log file and the timestamped log format
`monitor.py`	`watch()`	Per-service coroutine: reads metrics, checks the process, counts consecutive breaches
`recovery.py`	`recover()`	Stops the process, applies exponential back-off, restarts it or fails over to the standby

6.3 Complete Source Code

```
import asyncio, logging, subprocess, psutil
from dataclasses import dataclass
from logging.handlers import RotatingFileHandler

@dataclass
class Config:
    cpu_limit: float = 85.0       # percent
    mem_limit: float = 90.0       # percent
    interval: float = 2.0         # seconds between checks
    violations: int = 3           # consecutive breaches = fault
    max_restarts: int = 3
    base_delay: float = 1.0
    max_delay: float = 30.0

def setup_logger(path="faultguard.log"):
    log = logging.getLogger("faultguard")
    log.setLevel(logging.INFO)
    handler = RotatingFileHandler(path, maxBytes=1_000_000, backupCount=3)
    handler.setFormatter(logging.Formatter(
        "%(asctime)s | %(levelname)s | %(message)s"))
    log.addHandler(handler)
    return log

async def recover(service, cfg, log):
    if service.restarts >= cfg.max_restarts:
        log.critical("%s: restart limit reached, failing over",
                     service.name)
        service.proc = subprocess.Popen(service.standby_cmd)
        service.restarts = 0
        return
    service.restarts += 1
    delay = min(cfg.max_delay,
                cfg.base_delay * 2 ** (service.restarts - 1))
    try:
        service.proc.terminate()
        service.proc.wait(timeout=5)
    except subprocess.TimeoutExpired:
        service.proc.kill()
    await asyncio.sleep(delay)
    service.proc = subprocess.Popen(service.cmd)
    log.info("%s: restarted (attempt %d)", service.name,
             service.restarts)

async def watch(service, cfg, log):
    breaches = 0
    while True:
        try:
            cpu = psutil.cpu_percent(interval=None)
            mem = psutil.virtual_memory().percent
            dead = service.proc.poll() is not None

            if cpu > cfg.cpu_limit or mem > cfg.mem_limit:
                breaches += 1
                log.warning("%s: CPU=%.1f MEM=%.1f (breach %d)",
                            service.name, cpu, mem, breaches)
            else:
                breaches = 0

            if dead or breaches >= cfg.violations:
                log.error("%s: fault detected, recovering",
                          service.name)
                await recover(service, cfg, log)
                breaches = 0
        except Exception:
            log.exception("%s: monitor error", service.name)
        await asyncio.sleep(cfg.interval)
```

Listing 1: Complete source code of FaultGuard

6.4 Program Output

Every stage writes to the log in the format `time | level | message`. Figure 3 shows the log of a CPU-overload test: three consecutive breaches are logged as warnings, the fault is declared and the service is restarted. Figure 4 shows a service that crashes at every start: the restarts are delayed by 1, 2 and 4 seconds, and after the third restart the daemon logs a critical message and fails over to the standby instance.

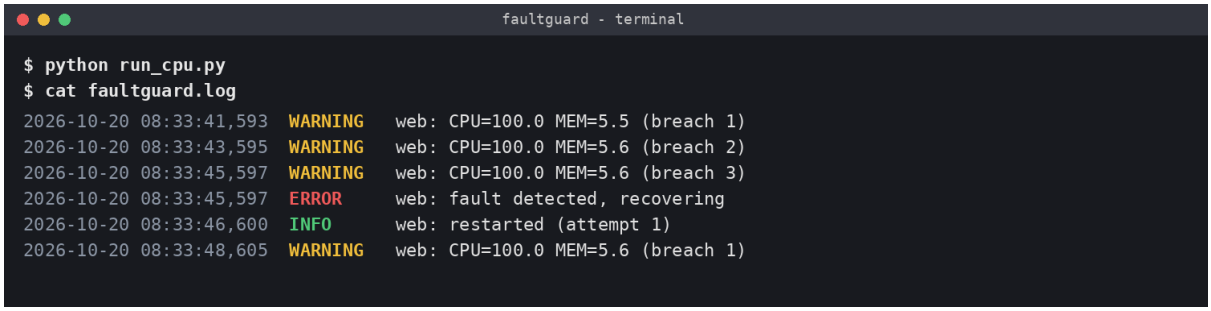


Figure 3: Log output for the CPU-overload test (breaches, fault, restart)

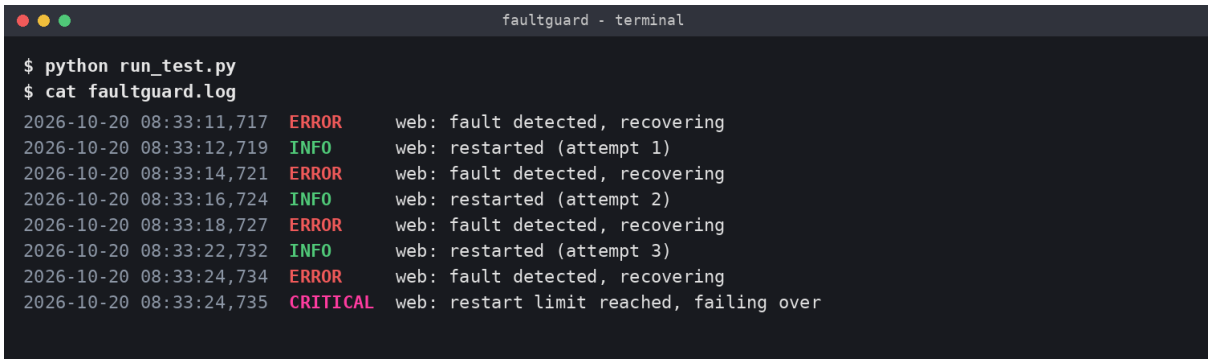


Figure 4: Log output for the repeated-failure test (back-off, then failover)

6.5 Testing and Validation

The system was validated with fault-injection tests, in which failures were created deliberately and the behaviour of the daemon was observed in the log file. Table 5 lists the test cases.

Table 5: Fault-injection test cases

No.	Scenario	Injected Fault	Expected Behaviour
1	Process crash	Monitored script exits with an error	Crash detected, logged, process restarted
2	CPU overload	Busy loop pushes CPU above the threshold	Warnings logged, restart after N breaches
3	Memory pressure	Script keeps allocating memory	Breach logged, service restarted
4	Short CPU spike	Spike shorter than N intervals	No restart (false alarm filtered)
5	Repeated failure	Service crashes at every start	Back-off grows, then failover to standby
6	Internal exception	Check function raises an error	Exception logged, daemon keeps running

7 Results and Discussion

7.1 Fault Detection and Recovery

In the fault-injection tests, FaultGuard detected every injected fault and carried out the matching recovery action. A crashed process was detected at the next polling interval and restarted automatically. Sustained CPU and memory overload was first reported as warnings and then resolved by a restart once the violation count was reached. A service that failed repeatedly was handled by exponential back-off followed by failover to the standby instance, which avoided a restart storm.

7.2 False-Alarm Filtering

Short spikes lasting fewer than N consecutive checks were correctly ignored. This shows that the consecutive-violation filter reduces false positives and prevents unnecessary restarts.

7.3 Resilience and Logging

Exceptions deliberately raised inside a health check were caught and recorded, and the daemon continued to run, which confirms that the monitor itself is resilient. The rotating log gave a complete timestamped trail of warning, error, critical and recovery events, so it was easy to see what happened and when.

7.4 Overhead and Tuning

Because monitoring uses a single `asyncio` loop, the daemon itself needed very little CPU and memory, even with several services under watch. The main trade-off remains the choice of polling interval and thresholds: shorter intervals reduce detection time but increase overhead, so these values should be tuned for each deployment.

8 Conclusion and Future Work

FaultGuard demonstrates how Python's subprocess management, exception handling, logging and asynchronous event loops can be combined into a practical, reliable system-level tool. It monitors CPU and memory thresholds, tolerates its own internal errors, records all events in a structured log and automatically restarts or fails over faulty services. The project also shows the engineering trade-offs involved in fault-tolerant design, such as detection speed against false alarms and availability against restart storms.

8.1 Future Work

- Monitoring disk, network and per-process resource usage.
- Statistical or machine-learning based anomaly detection instead of fixed thresholds.
- Alerts through e-mail, SMS or messaging platforms.
- A web dashboard showing live health status and historical logs.
- Monitoring multiple hosts and containers in a distributed deployment.
- An external watchdog that restarts FaultGuard itself.

8.2 Objectives Achieved

Table 6 relates the objectives stated in the Introduction to the outcomes obtained.

Table 6: Objectives versus outcomes

Objective	**Outcome**
Monitor CPU and memory	Host usage read with `psutil` at a configurable interval
Detect crashes and overload	Process exit and N consecutive breaches both detected in the fault-injection tests
Log events and exceptions	Timestamped, levelled log in a rotating file; no exception stopped the daemon
Recover and fail over	Restart with exponential back-off, then failover to the standby instance
Lightweight and crash-proof	Single `asyncio` loop with all checks inside `try/except`

8.3 Limitations

- Detection is polling-based, so a fault is noticed only at the next interval.
- Host-wide CPU and memory figures are used; per-process limits are not yet enforced.
- The daemon runs on a single host and is itself a single point of failure until an external watchdog is added.
- Thresholds are fixed values and must be tuned manually for each deployment.

9 Mapping of Program Outcomes (POs) and Sustainable Development Goals (SDGs)

9.1 Program Outcomes (POs)

Table 7: Mapping of Program Outcomes (POs) to the project

PO	Outcome	Contribution of the Project
PO1	Engineering Knowledge	Applies operating-system concepts, Python programming, concurrency and basic mathematics (exponential back-off) to build a working fault-tolerant monitoring system.
PO2	Problem Analysis	Identifies failure modes of long-running services, reviews dependability literature and formulates thresholds and recovery policies from the analysis.
PO3	Design/Development of Solutions	A modular daemon was designed with separate monitoring, detection, recovery and logging components, balancing availability, overhead and cost.
PO4	Conduct Investigations of Complex Problems	Fault-injection experiments were designed and the log data analysed to evaluate detection accuracy and recovery behaviour.
PO5	Engineering Tool Usage	Python, `asyncio`, `psutil` and the `logging` module were selected and used, with awareness of limits such as polling-based detection.
PO6	The Engineer and The World	Automatic recovery reduces downtime of services that society depends on, and resource monitoring avoids wasted computing resources and energy.
PO7	Ethics	Open-source libraries were used under their licences, all sources are acknowledged, and only system metrics (no personal data) are logged.
PO8	Individual and Teamwork	The work was shared among the three team members and integrated through regular discussion and testing.
PO9	Communication	The project is documented with diagrams, code listings and test results in this report and can be presented to technical and non-technical audiences.
PO10	Project Management and Finance	The project was planned in phases (requirements, design, implementation, testing) using free, open-source tools, keeping the cost minimal.
PO11	Life-Long Learning	The team studied asynchronous programming, process management and reliability engineering independently, which are fast-changing areas of software development.

9.2 Program Specific Outcomes (PSOs)

Table 8: Mapping of Program Specific Outcomes (PSOs) to the project

PSO	Outcome	Contribution of the Project
PSO1	Apply knowledge of programming, algorithms and system software to design and develop software solutions.	FaultGuard applies Python programming, process management and concurrency to build a system-level software solution.
PSO2	Use modern tools and technologies to develop reliable and efficient computing applications.	The project uses `asyncio`, `psutil` and structured logging to build a reliable, low-overhead monitoring application.

9.3 Sustainable Development Goals (SDGs)

Table 9: Alignment of the project with SDGs

SDG	Goal	Alignment
SDG 4	Quality Education	Serves as a practical learning project in Python, operating systems and reliability engineering.
SDG 8	Decent Work and Economic Growth	Reduces costly downtime and manual operations effort, improving the productivity of IT-dependent businesses.
SDG 9	Industry, Innovation and Infrastructure	Supports resilient digital infrastructure by keeping critical services available through automated fault detection and recovery.
SDG 12	Responsible Consumption and Production	Monitoring of CPU and memory helps find resource leaks and wasteful usage, promoting efficient use of computing resources.

Bibliography

[1] Beyer, B., Jones, C., Petoff, J., and Murphy, N. R., eds., *Site Reliability Engineering: How Google Runs Production Systems*, O'Reilly Media, 2016.

[2] Python Software Foundation, "asyncio – Asynchronous I/O," Python 3 Documentation. https://docs.python.org/3/library/asyncio.html

[3] A. Avizienis, J.-C. Laprie, B. Randell, and C. Landwehr, "Basic concepts and taxonomy of dependable and secure computing," *IEEE Transactions on Dependable and Secure Computing*, vol. 1, no. 1, pp. 11–33, 2004.

[4] J. Armstrong, *Making Reliable Distributed Systems in the Presence of Software Errors*, Ph.D. dissertation, Royal Institute of Technology (KTH), Stockholm, 2003.

[5] M. Lutz, *Learning Python*, 5th ed., O'Reilly Media, 2013.

[6] M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2nd ed., Pragmatic Bookshelf, 2018.

[7] Python Software Foundation, "subprocess – Subprocess management," Python 3 Documentation. https://docs.python.org/3/library/subprocess.html

[8] Python Software Foundation, "logging – Logging facility for Python," Python 3 Documentation. https://docs.python.org/3/library/logging.html

[9] G. Rodola, "psutil: Cross-platform library for retrieving information on running processes and system utilization." https://psutil.readthedocs.io